

ARES: Automated Reconciliation and Evidence for STIGs

Roman Di Domizio
Matthew Nguyen
Miguel Cerna Benitez

Agents:

ARES Parser Agent AIC 2026
ARES Reporting Agent AIC 2026
ARES Determination Agent AIC 2026
ARES Gap Coverage Agent AIC 2026

Introduction



- **Roman Di Domizio**
- Longmont, CO
- B.S. Computer Science
- M.S. Artificial Intelligence (in progress)
- Space, NSS: NGG, Cyber Security Engineer Intern
- Hockey, snowboarding, hiking, running



- **Matthew Nguyen**
- Houston, TX
- B.S. Computer Science
- Enterprise Operations: Software Engineer Intern
- Tennis, Running and Music



- **Miguel Cerna Benitez**
- Palmdale, CA
- B.A Computer Systems – web programming
- M.S National Cyber Security Studies (In progress)
- Aeronautics, ISSO Intern
- Member of National Security Studies Club

Problem Statement

Issue Identified

- Cybersecurity engineers at LM spend thousands of hours annually on Security Technical Implementation Guide's (STIGs)
- Automated STIG scanning tools (e.g., PreSTIGe and evaluate-STIG) make mistakes, do not have full coverage, have conflicting findings, and require manual verification or spot checks
- 41 disagreements (11.2 % of checks) between PreSTIGe and evaluate-STIG on one RHEL 8 VM
- 20 “Not Reviewed” gaps (checks missed by either the automated tools)
- 3–4 hours of manual reconciliation per system (SSH, run commands, compare outputs)
- Both tools missed CAT I critical issues (false negatives)

Why It Matters

- LM programs may involve hundreds-to-thousands of systems or endpoints to validate
- Manual verification defeats the purpose of using an automated scanner
- Random spot checks introduce risk
- Manual reconciliation requires a team of cyber engineers
- No POA&M comment generation → additional manual drafting needed
- Each not reviewed rule or disagreement triggers manual CLI validation

Solution Overview

Target Audience: Cybersecurity engineers and compliance auditors

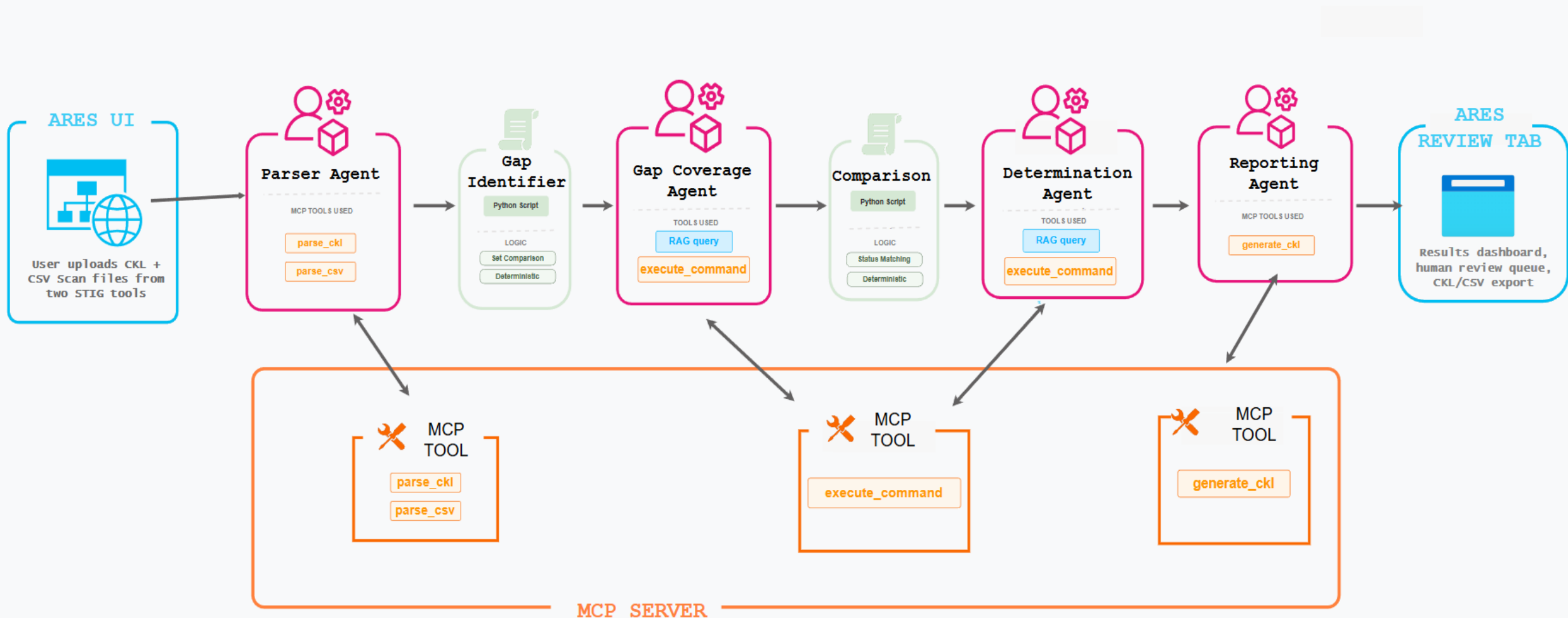
Key Features:

- Multi-tool parsing (CKL & CSV) → unified schema
- CLI access for autonomous validation - Agent doesn't just compare reports, it runs actual "check text" commands on the target system to determine ground truth.
- Disagreement resolution using RAG-enhanced reasoning
- Automated report generation (CKL, metrics, POA&M, recommendations)

Process Flow (4 Genesis agents + 2 Python scripts):

1. Upload CSV/CKL to Streamlit Frontend
2. Parser Agent (CKL/CSV to standardized JSON via MCP)
3. Gap Identifier Script (identifies "Not Reviewed" checks)
4. Gap Coverage Agent (validates "Not Reviewed" checks; SSH into VM via MCP + Fabric to run "Check Text" commands retrieved via RAG)
5. Comparison Script (identifies conflicting findings between the 2 automated scanners and/or Gap Coverage agent)
6. Determination Agent (validates conflicts; SSH into VM via MCP + Fabric to run "Check Text" commands retrieved via RAG)
7. Reporting Agent (generates final CKL based on agent findings)
8. Results Tab on Frontend → Human Review → Export

Project Architecture



Recorded Demo Link

[ARES-demo.mp4](#)

Value Proposition

Per System:

- Manual: 3-4+ hours
- With ARES: <30 minutes
- Savings: 2.5-3.5+ hours per system

NGG Scale (1000+ unclassified systems):

- Current: 3,500+ hours per cycle (1 cycle = full STIG validation across all systems)
- With ARES: 500-750 hours per cycle
- Saves: 2,750-3,000 hours per cycle

Annual Impact (2-3 cycles/year):

- Time savings: 5,500-9,000 hours
- Equivalent to: 2.6-4.3 FTEs freed up year-round
- Cost savings: \$286K-473K annually

Technical Details

- **Genesis AI Agents** Parser, Gap Coverage, Determination, Reporting
- **Genesis LLM Model:** NVIDIA nemotron-3-ultra-550b-a55b
- **Python Execution Framework:** MCP Server (format parsers), CLI Executor (Fabric SSH), Streamlit UI, Audit Logger, Security Validator
- **Data Flow:** Upload from Streamlit Frontend → Parser Agent → Gap Identifier Script → Gap coverage Agent → Comparison Script → Determination Agent → Reporting Agent → Results Tab on Frontend → Human Review → Export
- **Integrations:** RAG vector DB (RHEL 8 STIG XML), SSH target (**rhel8-stig-lab**), Model Context Protocol (MCP) connectors
- **Security:** Read-only SSH user, command blacklist, full audit trail, no CKLB support in MVP

Challenges & Lessons Learned

MCP tool deployment to Genesis via PCell OpenShift and Panel group access

- Longest project blocker, multiple deployment attempts
- Successfully resolved after securing Panel permissions

Ambitious scope for 14-day timeline

- Complex backend (MCP server, CLI executor, parsers)
- Custom UI with multi-agent orchestration
- Required strict prioritization and time management

Networking complexity across multiple layers

- OpenShift → Genesis → Streamlit → RHEL 8 VM
- Multiple configurations for seamless data flow

Lessons Learned:

- Plan for enterprise deployment access early
- Modular architecture enables parallel work
- Priorities critical for aggressive timelines

Future Improvements

Architecture:

- LLM model router (optimize speed vs. accuracy)
- Multi-tool support (OpenSCAP, Nessus)
- CKLB format support
- Windows and other OS/platform support

Agent Capabilities:

- Spot Check Agent (CAT I triple-validation)
- Fixing Agent (remediation scripts)
- STIG Explainer chatbot
- Compliance tracking and pattern learning

Fleet-Scale:

- Multi-system parallel processing
- Fleet intelligence and systemic issue detection
- Distributed CLI execution
- Automated scheduling

